CloudShield AI / DAIL

02 — System Architecture Specification

Version 1.0 | Final Architecture Baseline | 1 September 2026

Purpose: Define the complete software architecture of the CloudShield AI / DAIL prototype, including system boundaries, architectural layers, components, responsibilities, trust boundaries, data/control flows, interfaces, persistence, deployment, security boundaries, failure isolation, and architectural acceptance criteria. This document refines the approved Software Requirements Specification without redefining the research contribution.

Document Control

Field	Value
Document ID	CSAI-DAIL-ARCH-002
Status	FINAL ARCHITECTURE BASELINE
Parent document	01 — Software Requirements Specification v1.0
System	CloudShield AI with DAIL as the research core
Architecture style	Modular layered monolith with explicit domain/service boundaries; adapters at the edges
Initial execution	Local deterministic prototype
Cloud scope	AWS controlled MVP
IaC	Terraform/HCL controlled subset
Primary language	Python 3.12
Persistence	SQLite through persistence abstraction
Graph	NetworkX
LLM	Provider-agnostic adapter, introduced after deterministic core

1. Architectural Goals

- Keep DAIL's trust boundary explicit and enforceable in software.
- Separate candidate construction, analysis, verification, and trusted-state mutation.
- Make the deterministic core runnable without live LLM or AWS dependencies.
- Keep domain logic independent of storage, web framework, and LLM provider.
- Make every safety-critical decision traceable to inputs and evidence.
- Permit later LLM and experiment integration without restructuring the core.
- Make uncertainty a first-class architectural concern rather than an exception hidden in implementation.
- Keep MVP scope narrow enough that infrastructure semantics can be explicitly controlled and tested.

2. System Boundary

2.1 Product Boundary

```
CLOUDSHIELD AI
├── Detection / CSPM context
├── Context / Risk context
├── AI Remediation context
└── DAIL RESEARCH CORE
```

```
├── Candidate State Builder
├── Trusted State Manager
├── Protected Invariant State
├── Resource Identity Engine
├── Dependency Graph Engine
├── Impact / Invalidation Engine
├── Verification Engine
├── Promotion Controller
└── Evidence / Audit Pipeline
```

The broader CloudShield product can eventually contain additional capabilities, but the implementation described here treats DAIL as the scientific object under test. Generic CSPM functionality is not required for the DAIL MVP.

2.2 External Boundary

```
Outside DAIL:
LLM / model provider
Terraform CLI
AWS APIs
Developer / experiment controller
Independent oracle

Inside DAIL:
Candidate construction
Trusted state
Invariant state
Identity
Dependency
Impact
Verification
Promotion
Evidence
```

The LLM is outside the trust boundary because its output is untrusted. Terraform and AWS are infrastructure/tool adapters rather than authorities for promotion. The Promotion Controller is the authoritative trust-state mutation boundary.

3. Architectural Style

The recommended implementation is a modular layered monolith. This is deliberate: the research prototype does not need distributed services, queues, service meshes, or production microservice infrastructure. Components are separated logically and through Python interfaces, while initially running in one process/application.

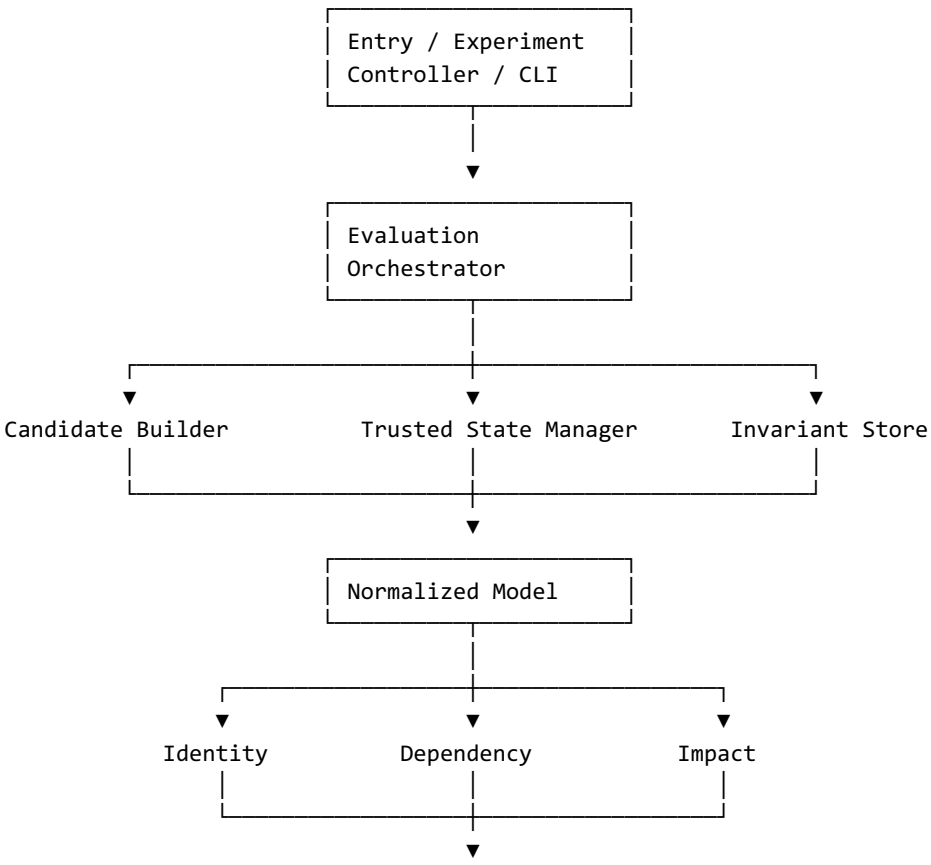
```
Presentation / CLI / future API
            ↓
Application Orchestration
            ↓
DAIL Domain Services
            ↓
Domain Models / Policies
            ↓
Ports / Interfaces
            ↓
Infrastructure Adapters
(Terraform / SQLite / AWS / LLM / filesystem)
```

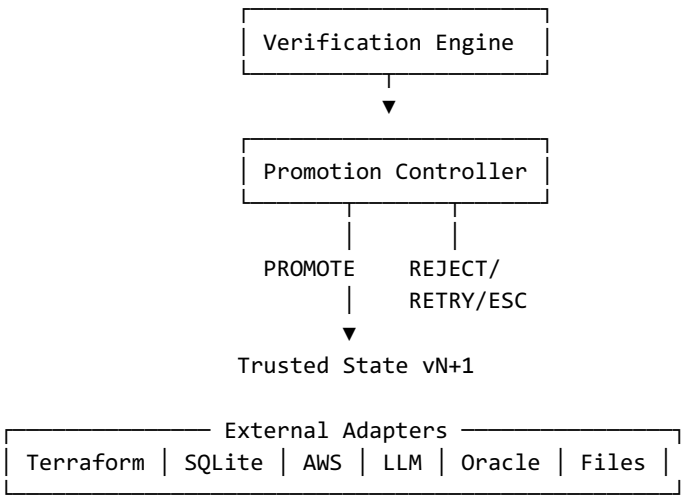
A later deployment may split components if required, but such a change must preserve the logical contracts and trust boundaries.

4. Layer Responsibilities

Layer	Responsibilities	Examples
Presentation/Entry	CLI, future API, experiment runner; converts external commands to application requests.	CLI, future FastAPI
Application	Coordinates a complete candidate-evaluation use case; controls transaction boundaries.	EvaluationOrchestrator
Domain Services	Perform identity, dependency, impact, verification, and promotion logic.	IdentityEngine, ImpactEngine
Domain Models/Policies	Represent states, resources, invariants, results, and legal transitions.	TrustedState, CandidateState, Invariant
Ports	Abstract external capabilities from domain logic.	TerraformPort, RepositoryPort, LLMPort
Adapters	Implement external technologies.	TerraformAdapter, SQLiteRepository, LLMProviderAdapter
Evidence	Cross-cutting recording of research-critical events.	EvidenceRecorder

5. Component Architecture





6. Component Specifications

6.1 Candidate State Builder

Constructs an isolated Candidate State from a Trusted State plus a patch. It is the controlled boundary through which proposed infrastructure enters DAIL.

- Must not mutate Trusted State.
- Must preserve parent trusted-state identity.
- Must validate patch structure before downstream processing.
- Must preserve patch/source provenance.
- Must create a new candidate for every new proposal.

6.2 Trusted State Manager

Owns versioned trusted-state persistence and controlled state transitions.

- Loads the current trusted baseline.
- Creates evaluation contexts.
- Associates invariant/evidence state.
- Commits promoted candidates atomically.
- Never decides whether a candidate should be promoted.

6.3 Invariant Store

Stores machine-verifiable security and functional obligations and their state/evidence associations.

6.4 Identity Engine

Determines resource correspondence and change classification using conservative supported rules.

6.5 Dependency Graph Engine

Builds a typed graph of supported Terraform references, infrastructure relationships, and invariant-semantic relationships.

6.6 Impact / Invalidation Engine

Maps changed resources/dependencies to protected obligations that may require current verification.

6.7 Verification Engine

Runs the required structural, security, and functional verification layers and returns explicit result semantics.

6.8 Promotion Controller

Evaluates the complete decision context and is the only component authorized to advance Trusted State.

6.9 LLM Adapter

Generates candidate remediation proposals after M8. It never receives authority to mutate trusted state.

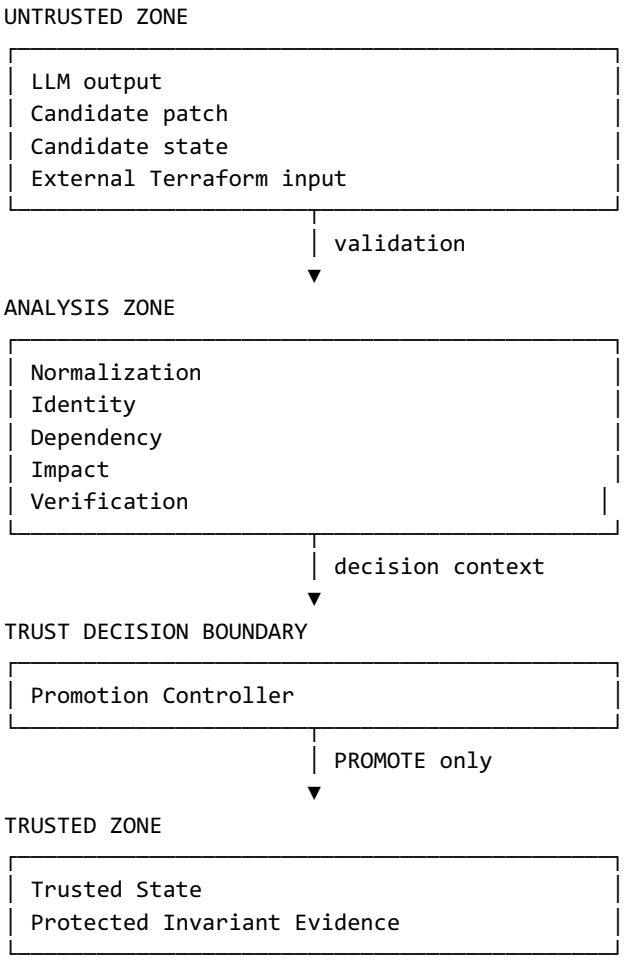
6.10 Evidence Pipeline

Captures the complete transition/evaluation chain in an auditable representation.

6.11 Independent Oracle

Evaluates outcomes independently from DAIL's own verifier/decision path and is primarily used for controlled experiments.

7. Trust Boundary Architecture

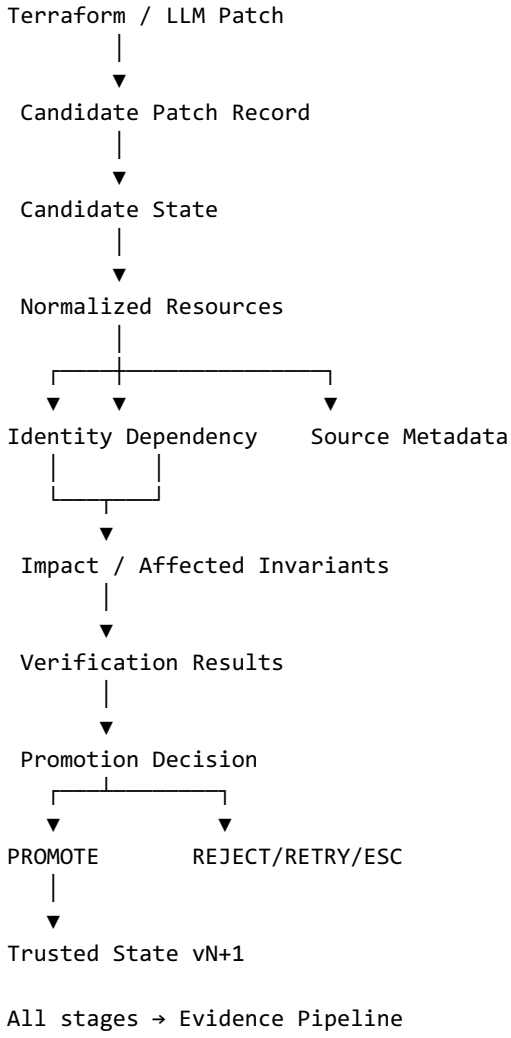


No external component, including the LLM, may cross directly into the trusted zone.

8. End-to-End Control Flow

1. Entry point receives a patch/evaluation request.
2. Trusted State Manager loads the current trusted baseline.
3. Candidate State Builder validates and constructs an isolated candidate.
4. Terraform adapter/normalizer creates the normalized candidate representation.
5. Identity Engine compares trusted and candidate resources.
6. Dependency Engine constructs the supported dependency graph.
7. Impact Engine determines affected protected obligations.
8. Verification Engine verifies required properties.
9. Promotion Controller consumes the complete decision context.
10. On PROMOTE, the state manager commits the candidate as the next trusted version.
11. On REJECT, RETRY, or ESCALATE, the current Trusted State remains unchanged.
12. Evidence Pipeline records the complete decision chain.

9. Data Flow Architecture



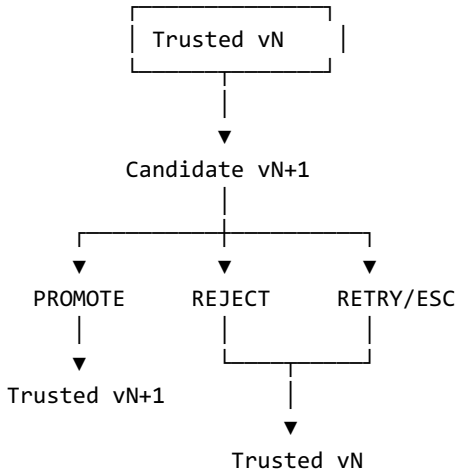
10. Candidate vs Trusted State Isolation

Property	Candidate State	Trusted State
Trust level	Untrusted	Trusted

Mutation	Immutable within evaluation attempt	Advanced only by Promotion Controller
Parent	References Trusted State	References previous trusted version when promoted
Use as next repair baseline	No	Yes
Verification	Pending/current candidate evaluation	Evidence already associated with trusted state
Failure effect	Discard/retry/escalate	Must remain intact

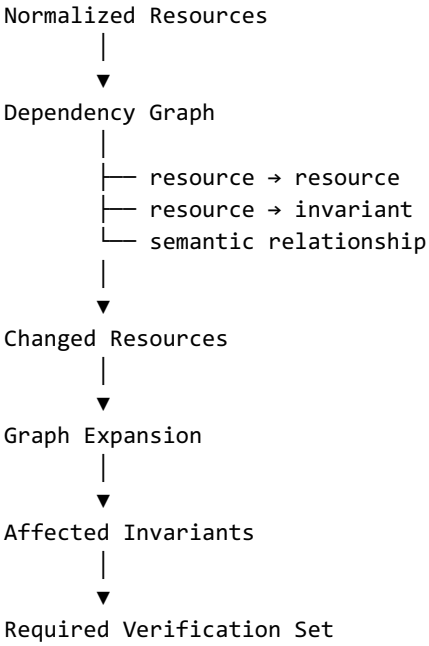
The architecture must make it difficult, not merely conventionally discouraged, for application code to mutate trusted state during candidate analysis.

11. State Transition Architecture



A rejected candidate does not become a parent trusted state. A retry begins from the existing trusted baseline unless the explicit research/implementation policy later defines otherwise.

12. Dependency and Impact Architecture



The dependency graph is intentionally a supported semantic subset. The architecture must expose unsupported/uncertain relationships rather than pretending the graph is complete.

13. Verification Architecture

```
VerificationEngine
├── StructuralVerifier
│    └── syntax / supported model / plan consistency
├── SecurityVerifier
│    └── MVP security predicates
├── FunctionalVerifier
│    └── selected reachability/function predicates
└── OptionalFormalVerifier
     └── only where explicitly justified
```

Each verifier produces typed results and evidence. The Promotion Controller consumes results; it does not secretly perform verification itself.

14. Promotion Architecture

```
Promotion Context
├── Candidate structural status
├── New requirement result
├── Affected invariant set
├── Current verification results
├── New-resource baseline results
├── Identity report
├── Dependency report
└── Blocking uncertainty/errors
            |
            ▼
     Promotion Policy
       |   |   |   |
       ▼   ▼   ▼   ▼
   PROMOTE REJECT RETRY ESCALATE
```

Promotion is a policy decision over explicit evidence. The controller is deliberately separate from individual verifiers to prevent one verifier from becoming an implicit promotion authority.

15. Interface/Dependency Rules

- Domain models must not import infrastructure adapters.
- Identity must not depend on Promotion Controller.
- Verification must not mutate state.
- Impact analysis must not declare invariants proven.
- Promotion Controller may consume all relevant reports but should not reimplement their internals.
- Evidence recording may observe every stage but must not change decision semantics.
- LLM adapters must depend on domain-neutral patch/request interfaces.
- Persistence repositories must be accessed through ports/interfaces from domain/application code.
- Entry points must orchestrate; they should not contain business rules.

16. Persistence Architecture

```
Application/Domain
      │
      ▼
Repository Ports
      │
      ▼
SQLAlchemy persistence adapter
      │
      ▼
SQLite

Logical stores:
States
Resources
Invariants
Evidence
Dependencies
Impact Results
Verification Results
Decisions
Runs
Attempts
LLM Events
Oracle Results
```

SQLite is the initial storage implementation. The architecture intentionally avoids SQLite-specific assumptions in domain logic so PostgreSQL can be introduced later without rewriting the core.

17. External Adapter Architecture

Adapter	Interface purpose	Initial implementation
Terraform Adapter	Validate/read controlled Terraform inputs and expose normalized source information.	Terraform CLI + limited HCL tooling
Storage Adapter	Persist/retrieve domain entities.	SQLAlchemy + SQLite
AWS Adapter	Controlled runtime queries/validation where required.	boto3
LLM Adapter	Generate untrusted remediation proposals.	Provider-specific implementation behind interface
Oracle Adapter	Run independent evaluation.	Controlled local/runtime oracle
Evidence Adapter	Persist research-critical evidence.	Database/file-backed implementation

18. Runtime Deployment Architecture

18.1 Initial Local Runtime

```
Developer machine
┌──────────────────────────────────────┐
│ Python DAIL application              │
│ ├── Domain services                  │
│ ├── Application orchestration        │
│ ├── SQLite                           │
```

```
│ └── Terraform CLI                    │
└──────────────────────────────────────┘
          │
          ├── optional AWS credentials
          └── optional LLM provider
```

M1–M8 should be runnable locally without requiring a live LLM. Live AWS is used only where a validation/oracle requirement actually needs it.

18.2 Controlled AWS Runtime

```
DAIL runner
   │
   ├── least-privilege AWS credentials
   │
   ▼
Controlled AWS account/environment
   ├── VPC
   ├── Subnet
   ├── Routing
   ├── EC2
   ├── Security Groups
   └── RDS
```

This is a test/validation environment, not a production deployment target for arbitrary generated infrastructure.

19. Security Architecture

Boundary	Control
LLM	Untrusted; adapter only; no state mutation access.
Terraform patch	Validated before candidate construction.
Candidate	Isolated from Trusted State.
Promotion	Single authoritative mutation boundary.
AWS	Controlled account/environment; least privilege.
Secrets	Environment/secret manager; never source-controlled.
Evidence	Redact sensitive values where appropriate.
Dependencies	Uncertainty explicitly represented.
Unsupported semantics	Cannot silently become proof.
Production	Prototype must not deploy arbitrary LLM-generated infrastructure to production.

20. Failure Isolation Architecture

```
External failure
   │
   ├── LLM error ──────────► attempt failure; trusted unchanged
   ├── Terraform error ────► candidate failure; trusted unchanged
   ├── AWS error ──────────► verifier error/unknown; policy handles
   ├── DB error ───────────► transaction failure; trusted unchanged
   └── verifier error ─────► VERIFIER_ERROR; no implicit PASS
```

Failures are represented explicitly and propagated to the Promotion Controller. A failure in an analysis component must not create an accidental path to promotion.

21. Concurrency and Consistency

The initial prototype can assume a single active evaluation process for a given experiment/run. Nevertheless, state consistency rules must be implemented as if candidate evaluations were isolated.

- Trusted State is read as a versioned baseline at candidate creation.
- Candidate evaluation operates against a specific parent trusted-state version.
- Promotion must verify that the expected parent is still current before committing.
- If the parent has changed, promotion must fail/retry rather than silently overwrite the newer trusted state.
- Evidence must identify the exact parent and candidate versions involved.

22. Observability Architecture

```
Operational logs
   └── debugging / runtime diagnostics

Research evidence
   └── immutable decision/evaluation chain

Both may share correlation IDs but have different purposes.
```

- Every run receives a run ID.
- Every repair attempt receives an attempt ID.
- Every candidate receives a candidate ID.
- Every trusted state receives a version/state ID.
- Every verification result references its evaluated state/candidate.
- Every promotion decision references its decision context.
- Verifier/configuration versions are recorded.

23. Architecture for A/B/C Experiment Modes

Mode	Architectural behavior
A — Stateless	Executes remediation without persistent DAIL protected-state behavior; retains required run evidence.
B — Stateful Full	Maintains persistent state/invariants and verifies the full relevant protected set according to the experimental policy.
C — DAIL	Uses persistent state, dependency-aware impact analysis, and selective re-verification.

The architecture should share infrastructure and execution plumbing where possible while keeping condition-specific policy explicit. The experimental implementation must not accidentally give one condition different model capability or hidden context.

24. Independent Oracle Boundary

```
                    DAIL path
Candidate ──────────► DAIL Verifiers ──────► DAIL Decision

Candidate ──────────► Independent Oracle ──► Ground Truth

No dependency:
```

```
Oracle ─X─► Promotion Controller
Oracle ─X─► DAIL verifier result
```

The oracle must remain logically independent enough that agreement with DAIL is meaningful. The exact oracle implementation is a later engineering/research decision and is not silently fixed by this architecture document.

25. Recommended Package Architecture

```
dail/
├── state/
│   ├── models.py
│   ├── service.py
│   └── repository.py
├── invariants/
│   ├── models.py
│   ├── store.py
│   └── lifecycle.py
├── terraform_model/
│   ├── models.py
│   ├── parser.py
│   ├── normalizer.py
│   └── fingerprint.py
├── identity/
│   ├── models.py
│   └── engine.py
├── dependency/
│   ├── models.py
│   ├── rules.py
│   └── engine.py
├── impact/
│   ├── models.py
│   └── engine.py
├── verification/
│   ├── models.py
│   ├── structural.py
│   ├── security.py
│   └── functional.py
├── promotion/
│   ├── models.py
│   ├── policy.py
│   └── controller.py
├── llm/
│   ├── models.py
│   ├── interface.py
│   └── adapters/
└── evidence/
    ├── models.py
    └── recorder.py
```

The exact file split may change during implementation, but the logical ownership boundaries shall remain stable unless a documented architectural decision changes them.

26. Architecture Decision Records Required

- ADR-001: Modular monolith rather than microservices for MVP.

- ADR-002: Python 3.12 and dependency management approach.
- ADR-003: Terraform CLI plus controlled normalization layer.
- ADR-004: SQLite through persistence abstraction.
- ADR-005: NetworkX for dependency graph.
- ADR-006: Provider-agnostic LLM boundary.
- ADR-007: Promotion Controller as sole trusted-state mutation authority.
- ADR-008: Explicit uncertainty result semantics.
- ADR-009: Deterministic fixed-patch core before LLM integration.
- ADR-010: Independent oracle separation.

27. Architecture Constraints

- The system must remain within the controlled AWS/Terraform MVP scope.
- Unsupported infrastructure semantics cannot be promoted as verified.
- Candidate evaluation cannot mutate trusted state.
- Promotion must remain centralized.
- Domain logic cannot become coupled to external providers.
- Experiment infrastructure cannot alter DAIL semantics merely for convenience.
- Architecture changes affecting research validity require documented review.

28. Architecture-to-Requirements Traceability

SRS requirement area	Architectural realization
FR-CAND	Candidate State Builder + isolated candidate model
FR-STATE	Trusted State Manager + repository boundary + versioning
FR-TF	Terraform Adapter + normalization layer
FR-ID	Identity Engine
FR-DEP	Dependency Graph Engine
FR-IMP	Impact / Invalidation Engine
FR-INV	Invariant Store + lifecycle
FR-VER	Verification Engine
FR-PROM	Promotion Controller
FR-LLM	LLM Adapter outside trust boundary
FR-EVID/DATA	Evidence Pipeline + persistence
SAFE/ERR	Trust boundary + centralized promotion + explicit result types
NFR-REP	Deterministic adapters, canonicalization, versioned evidence
NFR-TST	Layered modules, interfaces, local deterministic execution
NFR-SEC	Adapter boundaries, credential controls, candidate isolation

29. Architecture Acceptance Criteria

- Every major SRS functional area maps to an explicit component.
- Candidate and Trusted State are architecturally separated.
- LLM has no direct trusted-state mutation path.
- Promotion Controller is the only trusted-state mutation authority.

- Domain services do not depend on concrete infrastructure technologies.
- Terraform/AWS/LLM/DB access is isolated behind adapters or ports.
- Failure paths preserve Trusted State.
- Concurrent parent-state changes cannot silently overwrite newer trusted state.
- Research evidence can correlate every evaluation stage.
- The architecture supports M1–M15 without requiring a foundational redesign.

30. Architectural Non-Goals

- Do not build a production microservice fleet.
- Do not introduce Kubernetes, queues, service mesh, or distributed transactions for the MVP.
- Do not build a generic cloud abstraction layer.
- Do not implement a full Terraform interpreter.
- Do not implement all AWS network semantics.
- Do not place the LLM inside the trusted core.
- Do not make the UI/API a prerequisite for DAIL correctness.

31. Final Architecture Directive

The final architecture is a deterministic, modular DAIL core surrounded by explicit adapters for Terraform, persistence, AWS, LLMs, and independent evaluation. The central architectural property is controlled trust: candidate infrastructure can be analyzed freely, but only the Promotion Controller can advance Trusted State.

The implementation shall begin with the local deterministic path. Architecture complexity should be added only when a later milestone requires it. Any change that weakens the trust boundary, changes the experimental comparison, expands the semantic scope, or alters the meaning of verification requires explicit documented review.

Appendix A — Architecture Glossary

Term	Meaning
Trusted State	Current infrastructure state accepted as the baseline for subsequent repair.
Candidate State	Untrusted proposed state under evaluation.
Invariant	Machine-verifiable security or functional obligation.
Identity	Determination of correspondence between resources across states.
Dependency	Supported relationship used to reason about impact.
Impact	Set of protected obligations potentially affected by a transition.
Verification	Evidence-producing evaluation of required properties.
Promotion	Controlled transition from Candidate State to Trusted State.
Oracle	Independent evaluation used to establish ground truth for experiments.
Adapter	Boundary implementation connecting DAIL to an external technology.

Appendix B — Architecture Source Baseline

This specification is grounded in the project's Software Requirements Specification v1.0 and the previously established Technical Software Engineering baseline. The baseline defines DAIL as the research core inside CloudShield AI, the deterministic-first implementation order, the supported Terraform/AWS scope, the explicit trust boundary, the component responsibilities, and the M1–M15 roadmap. filecite turn6file1
filecite turn6file15